

ASSIGNMENT-2

Pragnya

July 1, 2026

PART I: SHORT ANSWER/REASONING QUESTIONS

Q1: The above query would always return 0 since comparing to an unknown quantity(NULL) would not evaluate to TRUE but rather remain UNKNOWN. Instead, we can write it this way:-

```
SELECT COUNT(*)  
FROM Student  
WHERE phone_number IS NULL;
```

Also, aggregate functions like SUM, AVG, and COUNT(column) skip NULL values in computations to avoid mathematical distortion. Because NULL represents an UNKNOWN data point rather than a 0, treating it as zero would incorrectly affect averages.

Q2: Yes, "DISTINCT" keyword is actually redundant here since "GROUP BY" already divides them into distinct classes based on department. Here, "DISTINCT" just does extra computational work evaluating unique data, changing absolutely nothing about the final output.

Q3: The difference in row counts indicates that there are records in the Customer table that do not have any corresponding records in the Order table. The scenario that causes LEFT JOIN to return more rows occurs when one or more customers have placed zero orders. A LEFT JOIN preserves all records from the left table (Customer) even if no matching order exists, filling the missing order attributes with NULL values. This preservation of unmatched entities results in a higher overall row count.

Q4: Above Query is invalid in standard SQL since "GROUP BY" clause groups the entire dataset into distinct groups based on department so next when we select department and employee.name it would expect a single value but a particular department could have multiple employee names so it fails as it doesn't know which one to choose. We correct this using an aggregate function- GROUP_CONCAT.

Corrected code:

```
SELECT department, GROUP_CONCAT(employee_name), AVG(salary)
FROM Employee
GROUP BY department;
```

If standard SQL were to hypothetically allow this query to run as-is, displaying a single value for employee_name would force the database engine to make an arbitrary, non-deterministic choice.

Q5: The Query would result in an error as we are trying to use an aggregate function inside the WHERE clause. This occurs because of the execution order:

FROM → WHERE → GROUP BY → HAVING → SELECT

Corrected code:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6: a) Non-correlated subquery is more appropriate. The requirement asks for a comparison against the overall average price of all products. Since this global average is a single value that is completely independent of whichever row is currently being evaluated on the outer conveyor belt, the subquery only needs to execute exactly once at the beginning.

b) Correlated subquery is more appropriate because in order to evaluate an employee, the inner query must know that specific employee's department ID. Because the average salary changes from one department to another, the subquery must reference the outer query's row and recalculate the average row-by-row. When the outer table is large, row by row execution of inner loop which is dependent on outer table becomes slow and requires high CPU utilization.

PART II: LONG ANSWER QUESTIONS (CONTINUED)

Q7(a):

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Order o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

We use a LEFT JOIN to keep all customer rows regardless of their order history. By checking WHERE o.order_id IS NULL, we filter out any customers who matched with an order, isolating only the ones who have never placed an order.

Q7(b):

```
SELECT r.restaurant_id, rname, AVG(order_totals.
total_value) AS avg_order_value
```

```

FROM Restaurant r
JOIN (
    SELECT o.restaurant_id, o.order_id, SUM(mi.price *
        oi.quantity) AS total_value
    FROM Order o
    JOIN OrderItem oi
    ON o.order_id = oi.order_id
    JOIN MenuItem mi
    ON oi.item_id = mi.item_id
    GROUP BY o.restaurant_id, o.order_id
) AS order_totals
ON r.restaurant_id = order_totals.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(order_totals.order_id) > 5;

```

We first calculate the total value of each order using an inner subquery. Then, we take the average of these dynamic table results grouped by restaurant, filtering the final groups using HAVING to only include restaurants with an order count greater than 5.

Q7(c):

```

SELECT r.restaurant_id, r.name, AVG(order_totals.
    total_value) AS avg_order_value
FROM Restaurant r
JOIN (
    SELECT o.restaurant_id, o.order_id, SUM(mi.price *
        oi.quantity) AS total_value
    FROM Order o
    JOIN OrderItem oi
    ON o.order_id = oi.order_id
    JOIN MenuItem mi
    ON oi.item_id = mi.item_id
    GROUP BY o.restaurant_id, o.order_id
) AS order_totals
ON r.restaurant_id = order_totals.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING AVG(order_totals.total_value) > (
    SELECT AVG(mi.price * oi.quantity)
    FROM OrderItem oi
    JOIN MenuItem mi
    ON oi.item_id = mi.item_id
);

```

A non-correlated subquery is appropriate here because the platform-wide average order value is a single value computed once across the whole database. We place this static subquery inside the HAVING clause to filter out the restaurants.

Q7(d):

```

WITH ItemQuantities AS (
    SELECT mi.restaurant_id, mi.item_id, mi.item_name,
           SUM(oi.quantity) AS total_qty,
           RANK() OVER (PARTITION BY mi.restaurant_id ORDER
                        BY SUM(oi.quantity) DESC) AS rnk
    FROM MenuItem mi
    JOIN OrderItem oi
    ON mi.item_id = oi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.
           item_name
)
SELECT restaurant_id, item_name, total_qty
FROM ItemQuantities
WHERE rnk = 1;

```

We use a window function `RANK()` partitioned by `restaurant_id` to track item popularity. A limitation of basic `GROUP BY + MAX` approaches for top-N tracking is their inability to handle mathematical ties cleanly or safely retain unrelated row-level columns (like `item_name`) without throwing mismatch errors or omitting duplicate top ranks.

PART III: NORMALIZATION

Q1: `student_id` $\rightarrow$ `student_name`

`course_id` $\rightarrow$ `course_name`, `instructor_id`

`instructor_id` $\rightarrow$ `instructor_name`, `instructor_office`

`student_id`, `course_id` $\rightarrow$ `grade`, `enrollment_date`

Q2: Yes. The table is in 1NF because every cell holds just one single piece of data with no messy comma-separated lists, and the combined primary key (`student_id`, `course_id`) uniquely tracks every row.

Q3: The problem columns are `student_name` and `course_name` because they only care about one piece of the primary key instead of the whole thing. For example, if the university creates a new course, you cannot save it in the system until a student actually signs up for it, because you cannot leave the `student_id` blank.

Q4: The issue here is a chain reaction where `course_id` determines the `instructor_id`, which then determines the `instructor_office`. This causes a massive headache if an instructor moves to a new room because you have to hunt down and update every single student enrollment row for their classes. If you miss even one row, your database will claim the instructor is sitting in two different offices at the same time.

Q5: Student(`student_id`, `student_name`)

Course(`course_id`, `course_name`, `instructor_id`)

Instructor(`instructor_id`, `instructor_name`, `instructor_office`)

Enrollment(`student_id`, `course_id`, `grade`, `enrollment_date`)

PART IV: DESIGN JUSTIFICATION

For the first query in Part II, using a `LEFT JOIN` with a `NULL` check is much cleaner and more direct than a complex subquery. It helps the database process the rows smoothly without getting bogged down by subquery lookup limits. A `LEFT JOIN` naturally keeps the missing records visible as explicit blank slots, making it incredibly simple to filter out exactly who has never interacted with the platform.

Breaking the enrollment table down into 3NF highlights the classic trade-off between keeping data clean and making it easy to query. While normalization guarantees you will never accidentally corrupt your records during an update, it scatters your information across four different tables. Because of this, pulling a basic summary—like showing a student’s name next to their teacher’s office room—now requires stitching multiple tables back together using three separate `JOIN` operations. In the real world, developers often leave tables slightly messy or combined on purpose just to skip these heavy joins and keep read speeds lightning fast.